

ARGOS

Sprint Manual — Sebastian Montenegro

ML Layer 2 + LLM Triage Layer 4

Curso: Tópicos Avanzados de Ciberseguridad · Universidad de Lima ·
2026-1

Entrega final: 13 de junio de 2026

Versión 2.0 — manual organizado por orden de implementación (Fase 1 →
4)

Parte I — Introducción común al proyecto

ARGOS — Introducción al proyecto y arquitectura

Documento común a todos los manuales de integrante. Contiene contexto que cada miembro del equipo (P1/P2/P3/P4) necesita antes de leer su manual individual.

Field	Value
Type	Manual de introducción común para los 4 integrantes
Status	Active
Última actualización	2026-05-24
Pre-requisito	Ninguno. Léelo de cero.

1. Qué es ARGOS en una página

ARGOS es la **Adaptive Response Guard with Orchestrated Surveillance** — una plataforma multi-vector de detección y respuesta a amenazas que replica la arquitectura de productos comerciales high-end EDR/XDR (Microsoft Defender XDR, CrowdStrike Falcon, Palo Alto Cortex XDR) usando exclusivamente componentes open source más una API LLM de bajo costo para la capa de triage.

El proyecto es para el curso **Tópicos Avanzados de Ciberseguridad** en la Universidad de Lima · 2026-1. La entrega final es el **13 de junio de 2026** y consta de tres deliverables obligatorios: informe técnico (~30% del peso), demo en vivo (~40%) y presentación (~20%) más los seguimientos en semanas 5/7/9 (~10%).

El activo defendido es una base de datos **PostgreSQL 15** corriendo sobre una Linux VM en el lab. Tiene esquema `argos_demo_prod` con tablas que simulan datos de RRHH y finanzas (employees, payroll, customers, invoices, payments) con datos sintéticos. El host está tagged en Wazuh como `criticality=production-critical`, lo que dispara la regla de dos personas (two-person rule) para cualquier acción de containment sobre él.

El alcance multi-vector (post ADR-0008) cubre tres categorías de ataques distintas:

- **Ransomware:** cifrado de archivos via técnicas T1486/T1490/T1083/T1562. Es el énfasis primario.

- **Network Denial of Service:** ataques de red contra el puerto del servicio (T1498/T1499) con `hping3` o `slowhttptest`.
- **Application Abuse:** SQL injection contra una app web delante de la DB (T1190) usando `sqlmap`, y false positives de actividad legítima de usuario (T1078 Valid Accounts) — un SELECT masivo a las 3 AM se parece a exfiltración pero puede ser un reporte mensual legítimo.

Por qué este alcance y no más: ARGOS no busca cobertura exhaustiva multi-vector. Busca cobertura mínima representativa que demuestre que la arquitectura de 4 capas + SOAR + HITL es genuinamente adaptativa. Cubrir 3 vectores con profundidad real es más defendible que cubrir 10 superficialmente.

2. Arquitectura de 4 capas defensivas

ARGOS sigue el patrón industrial estándar de **defense-in-depth**: cuatro capas independientes y paralelas, cada una con trade-offs distintos, fallan independientemente. Si una capa cae o es evadida, las otras tres mantienen cobertura. No hay un solo punto de falla en la detección.



Capa 1 — Rule-Based Detection (Sigma + Wazuh)

Dueño: P3 (Angeles). Reglas YAML escritas en formato Sigma, convertidas a Wazuh con `sigma-cli`, mapeadas explícitamente a técnicas MITRE ATT&CK. Detectan patrones conocidos.

Sub-categorías por dominio (post ADR-0008):

- `detection/sigma-rules/ransomware/` — vssadmin, T1486 cifrado, T1083 enumeración, T1562 disable defender, T1021 lateral, T1071 C2
- `detection/sigma-rules/network/` — rate-based rules para T1498/T1499 (DDoS, slow-rate)
- `detection/sigma-rules/database/` — query pattern anomalies para UC-07
- `detection/sigma-rules/webapp/` — T1190 SQL injection signatures

Trade-off honesto: alta precisión, recall limitado contra variantes nuevas. Por eso existe Capa 2.

Capa 2 — ML Anomaly Detection

Dueño: P2 (Sebastian). Modelos no supervisados entrenados sobre baseline benigno. Detectan desviaciones que las reglas no captan.

Modelos especializados por dominio (post ADR-0008):

- `ml/models/ransomware_ensemble.pkl` — Isolation Forest + One-Class SVM. Features ventana 60s: file_write_rate, avg_entropy (Shannon), extension_modification_ratio, crypto_api_calls, new_outbound_connections, cpu_burst_score, io_burst_score.
- `ml/models/network_traffic_anomaly.pkl` — para UC-06 DDoS. Features: connections/sec, packet rate, source IP entropy, packet size variance.
- `ml/models/query_pattern_anomaly.pkl` — para UC-07 SELECT masivo. Features: rows_returned, query_duration_ms, hour_of_day, user_id, query_template_hash.

Ensemble ransomware: $\text{ensemble_score} = 0.6 \times \text{isolation_forest_score} + 0.4 \times \text{one_class_svm_score}$. Los demás modelos retornan score único.

Trade-off honesto: mayor recall contra variantes nuevas, mayor false positive rate. Requiere baseline limpio.

Capa 3 — Canary Deception

Dueño: P3 (Angeles). Archivos-cebo (`financials_Q4_2025.xlsx` , `passwords.txt` , `db_backup.sql`) en ubicaciones donde un usuario legítimo nunca tocaría. FIM (File Integrity Monitoring) con Sysmon whodata (Windows) o auditd (Linux) captura QUÉ proceso accedió.

Lógica: primera modificación / lectura / rename de un canary = alerta crítica con confianza máxima. Por diseño, FP rate ≈ 0 .

Esta capa es estrictamente ransomware-specific. No tiene sub-categorías por dominio. Documentado deliberadamente en ADR-0008: defender contra DDoS o SQL injection requiere otras primitivas, no canaries.

Capa 4 — LLM-Assisted Triage

Dueño: P1 (Enzo). FastAPI service que recibe el contexto completo de una alerta (process tree, network connections, file modifications) y produce análisis estructurado: técnica MITRE, severidad, runbook NIST aplicable, acción recomendada, IoCs a correlar.

Backend vendor-agnostic (per ADR-0001 v2):

- **Primary:** OpenAI GPT-4o-mini (US-based, soberanía de datos aceptable)
- **Fallback:** Llama 3.1 8B local vía Ollama (zero-egress, funciona sin internet)

Invariante crítico R-02: el LLM **NUNCA** está en el **path crítico de containment**. El SOAR decide desde Capas 1-3 solamente. El LLM enriquece la vista del analista; si halucina, miente, o cae, el sistema sigue funcionando.

Pipeline RAG: BM25 (léxico) + BGE-large embeddings (denso) + RRF (Reciprocal Rank Fusion). Hybrid retrieval estándar industrial. Sin cross-encoder reranker (descartado en ADR-0001 v2 por marginal gain vs costo).

3. SOAR Decision Engine y los 4 tiers de confianza

El **SOAR (Security Orchestration, Automation and Response)** es el cerebro que fusiona señales de las 4 capas y decide la acción. Lo dueño es **P1**. Implementa la lógica de tiers per ADR-0003.

Tier	Disparado por	Confianza	Acción
T0	Canary solo, OR las 3 capas corroboran	≥ 0.95	Aislamiento automático inmediato + snapshot
T1	Layer 1 + Layer 2 corroboran (sin canary)	0.80–0.95	Aislamiento automático inmediato + snapshot
T2	Capa sola con score alto	0.60–0.80	Throttle + snapshot ahora , aislamiento full pendiente de aprobación 3-min
T3	Corroboración baja	0.40–0.60	Solo notificación enriquecida con LLM

(*) **Los thresholds son preliminares** pendientes de calibración empírica per `OPEN_QUESTIONS_RESOLUTION.md` §Q5.

Tier T2 — el más interesante

Ransomware moderno cifra ~25,000 archivos/min. Un countdown de 3 minutos para aprobación humana sin protección parece estúpido. Por eso T2 NO es "pendiente": es "throttle + snapshot AHORA, full isolation pendiente de aprobación". El throttle (cpulimit/ionice) reduce la tasa de cifrado a ~100-500 files/min mientras los aprobadores ven el contexto LLM y deciden. Si el timeout de 3 min expira sin respuesta, el sistema auto-ejecuta (no espera más). Si el aprobador clickea Reject, el throttle se levanta y el caso queda registrado como false positive con razón documentada — esto es UC-07.

Split-brain (conflicto multi-aprobador)

Per ADR-0006: cuando hay 4 aprobadores y dan respuestas contradictorias (2 approve, 1 reject, 1 timeout), se aplica **conservative-wins policy** con ventana de consolidación de 60 segundos: cualquier "approve" gana sobre rechazos, excepto para acciones irreversibles que requieren **two-person rule** (dos approves explícitos, un reject cancela).

El **two-person rule** se activa cuando el host afectado tiene `criticality=production-critical` (nuestro PostgreSQL). Esto es UC-04 en el demo.

Canales de notificación (per ADR-0007 v2)

Tiempo	Canal	Propósito
t=0	Telegram bot con botones inline JWT	Primario — push agresivo a celulares de aprobadores
t=0	Discord webhook con @-mention de role	Visibilidad en server del equipo, presión social
t=60s	Twilio Voice DTMF (sin STT)	Escalación si nadie respondió. "Presione 1 para aprobar, 2 para rechazar"
post-decisión	Email	Resumen asíncrono, NUNCA en path crítico

4. Equipo y responsabilidades

	Integrante	Rol	Alcance principal
	Enzo Ordoñez Flores	P1 · Líder · LLM/ SOAR	<code>argos_contracts</code> (entregado), Capa 4 LLM Triage, motor SOAR + Tier Classifier, Approval API con JWT, notificaciones multi-canal, Consola de Aprobación Streamlit, simulador de ransomware, playbooks de containment, coordinación
	Sebastian Montenegro	P2 · Ingeniero ML	Capa 2 completa (Isolation Forest + One-Class SVM ensemble + modelos especializados), feature extraction, calibración thresholds, métricas P/R/F1, captura forense
	Angeles Castillo	P3 · Detección y Engaño	Capa 1 (Sigma rules ransomware + network + database + webapp), Capa 3 (canary FIM + whodata), validación con Atomic Red Team y Caldera, bonus: PRs upstream a SigmaHQ
	Diego Jara	P4 · Infraestructura	Vagrantfile + Wazuh/OpenSearch/Redis deployment, PostgreSQL con datos sintéticos, simuladores (ransomware + DDoS + SQL injection), UI Streamlit base, video demo

Regla operativa crítica: cada integrante debe poder defender su capa en exposición. P1 no escribe código de otros con Claude Code. Cada integrante puede usar Claude Code (o cualquier asistente IA) como **acelerador en SU propia parte**, siempre que entienda lo que produce y pueda defenderlo en viva (per CONTEXT.md §5 reinterpretado post ADR-0008).

5. Los 8 use cases del demo

El demo en vivo cubre 8 escenarios (per ADR-0008 multi-vector). Cada UC tiene narrativa específica que demuestra una propiedad distinta del sistema.

UC	Vector	Tier	Capas que firing	Qué demuestra
UC-01	Ransomware (LockBit-like)	T0	1+2+3	Full-stack end-to-end. Las 3 capas firing simultáneo. Auto-isolate + email post-facto.
UC-02	Canary deception	T0	3 sola	Detección ultra-temprana. Zero archivos reales cifrados.
UC-03	Variante novel + split-brain	T2	2 sola	Centerpiece HITL ransomware. ML atrapa lo que reglas no. 4 aprobadores. Conservative-wins live.
UC-04	PostgreSQL + two-person rule	T1	1+2	Compliance vocabulary. Four-eyes principle. Governance.
UC-05	Stealth attack (agent-kill)	T0	1+3+heartbeat	Resiliencia: agent disconnect = signal.
UC-06	DDoS volumetric	T0	1 (rate)+2 (network)	Cobertura multi-vector. Defiende contra ataques de red, no solo endpoint. T1498.
UC-07	SELECT masivo legítimo FP	T2	2 sola	Pieza clave del HITL. Humano cancela contención por FP reconocido. T1078.
UC-08	SQL injection	T1	1+2	OWASP Top 10 #1. T1190 Exploit Public-Facing Application.

Los dos son los UCs que más venden el HITL. UC-03 muestra split-brain con conservative-wins; UC-07 muestra cancelación de FP por humano. Ambos son lo que diferencia un EDR/XDR con HITL de un SIEM tradicional.

6. Ejemplo end-to-end de UC-01 paso a paso

Para que tengas un modelo mental concreto de cómo fluye una alerta a través del sistema, aquí está UC-01 (ransomware clásico LockBit-like) desglosado segundo a segundo.

Setup pre-ataque (T-5 min)

- Lab está corriendo: Wazuh manager + Windows VM + Linux VM con PostgreSQL.
- Los 4 aprobadores tienen Telegram abierto en sus celulares.
- La pantalla del proyector muestra: Streamlit Approval Console (vacía) + terminal de P4 listo para ejecutar el simulador.
- P1 narra el contexto: "Este es nuestro endpoint Windows típico de la organización. Tiene documentos en `C:\Users\Demo\Documents\`. El atacante acaba de obtener acceso inicial."

T=0 — Atacante ejecuta

P4 corre:

```
python attack-simulation/ransomware_simulator/lockbit_like.py --target windows-victim --speed full
```

Esto inicia el simulador. Behavior chain:

1. T+1s: enumera archivos en `C:\Users\Demo\Documents\` (técnica T1083 File and Directory Discovery)
2. T+2s: invoca `vssadmin delete shadows /all /quiet` (técnica T1490 Inhibit System Recovery)
3. T+3s: comienza a cifrar archivos con AES-256, renombrando a `.locked` (técnica T1486 Data Encrypted for Impact)
4. T+4s: toca el canary file `financials_Q4_2025.xlsx`
5. T+5s: deja la ransom note `README_RESTORE_FILES.txt` en el desktop

T+1 a T+5 segundos — Capas detectan en paralelo

Capa 1 (Sigma + Wazuh) detecta:

- T+2s: regla `T1490_vssadmin_delete_shadows` dispara cuando ve el comando `vssadmin`.
- T+3s: regla `T1486_mass_file_encryption_extension` dispara al ver renames masivos a `.locked`.
- Wazuh manager normaliza estas alertas y las publica al stream Redis `wazuh:alerts`.

Capa 2 (ML) detecta:

- ML consumer (Python proceso corriendo en bg) lee el stream Redis.
- Cada 60s genera features para los procesos activos. La ventana T+1 a T+60 captura el simulator process.
- Features observadas: `file_write_rate=347` archivos/min, `avg_entropy=7.8` (alta), `extension_modification_ratio=0.95`, `crypto_api_calls=42`, `cpu_burst_score=0.9`.
- Isolation Forest score: 0.94. One-Class SVM score: 0.91. Ensemble: 0.93.
- Publica `MLScore` con score 0.93 al stream Redis `ml:scores`.

Capa 3 (Canary FIM) detecta:

- T+4s: el simulador toca `financials_Q4_2025.xlsx`.
- Sysmon whodata captura el evento con PID del simulador y command-line completo.
- Wazuh dispara regla custom `canary_file_accessed` con severity 12 (crítica).
- Score implícito: 1.0 (canary fire = certeza máxima).

T+5 segundos — SOAR Decision Engine fusiona

El **SOAR orchestrator** (proceso de P1 corriendo el FastAPI Approval API en port 8003) se entera de las 3 alertas dentro del mismo incident window de 5 segundos.

Tier Classifier (`soar/decision_engine/tier_classifier.py`) aplica reglas:

- `canary_fired=True` → tier T0 (canary alone wins to T0).
- Pero también L1 + L2 corroboran con high scores → tier T0 confirmed.
- **Decisión:** `Tier.T0` con confidence 0.95+.

State machine (`soar/decision_engine/state_machine.py`):

1. Crea Incident con `incident_id=INC-2026-05-26-001`, state `RECEIVED`.
2. Persiste en Redis con key `incident:INC-2026-05-26-001`.
3. Llama al LLM Triage en paralelo (Capa 4 enrichment-only, NO bloquea decisión).
4. Transición de estado: `RECEIVED` → `PENDING_EXECUTION`.

Capa 4 LLM Triage (en paralelo, T+5 a T+8s):

- FastAPI `/triage` endpoint recibe el AlertContext con las 3 alertas.
- Llama a OpenAI GPT-4o-mini con prompt enriquecido.
- Devuelve TriageResponse: `tecnica_mitre=T1486`, `confianza=0.94`, `severidad=critical`, `runbook_aplicable="NIST 800-61 §3.4 Containment"`, `accion_recomendada="Isolate host immediately and capture memory snapshot before remediation"`.
- El Incident se actualiza en Redis con el `llm_analysis` campo.

T+8 segundos — Playbook automático

Como es T0 sin override de criticality (es una Windows VM standard, no production-critical), no requiere approval. El SOAR ejecuta el playbook directamente:

1. **Host isolation** (`soar/playbooks/host_isolation.py`): conecta vía PowerShell al Windows VM y crea regla firewall: `New-NetFirewallRule -DisplayName "ARGOS-Isolation" -Direction Outbound -Action Block`.
2. **Process kill**: identifica el PID del simulador desde el whodata FIM, ejecuta `Stop-Process -Force -Id <PID>`.
3. **Disk snapshot**: invoca Volume Shadow Copy `vssadmin create shadow /for=C:` para preservar evidencia forense.

4. Transición de estado: `PENDING_EXECUTION` → `EXECUTED` .

T+10 segundos — Notificación post-facto

El SOAR envía notificación multi-canal (per ADR-0007 v2):

- **Telegram** a los 4 chat_ids configurados: mensaje con incident_id, técnica, análisis LLM, y botón inline "Revertir" firmado con JWT.
- **Discord** webhook al server del equipo con embed coloreado por tier (T0 = rojo) y `@argos-approvers` mention.
- **Email post-facto** a `APPROVER_EMAILS` (los 4 emails del equipo) con resumen + link al audit log en OpenSearch.

T+10 a T+30 segundos — Streamlit Console muestra

La **Streamlit Approval Console** (proceso de P1 corriendo en port 8501, proyectado en pantalla) refresca cada 2 segundos vía `streamlit-autorefresh` . Muestra:

- **Panel izquierdo (Incident Card):** badge T0 en rojo, incident_id, host afectado, técnica MITRE T1486, análisis LLM compacto.
- **Panel central (Decision Matrix):** vacío porque es T0 auto-execute, no requirió aprobación.
- **Panel derecho (System Logic):** estado actual `EXECUTED` , decisión `EXECUTE_ISOLATION` , política aplicada `auto-execute` , justificación "T0 confirmed: canary + L1 + L2 corroborate".
- **Panel inferior (Action Timeline):** línea de tiempo horizontal mostrando: alert created → tier classified → playbook executed → notifications sent.

T+30 segundos — Análisis forense visible

P1 narra: "El sistema aisló el host en menos de 10 segundos. 31 archivos cifrados de 500. El canary salvó los restantes. Los 4 aprobadores tienen el incidente en su Telegram con la opción de Revertir si fuera falso. Audit log completo en OpenSearch con cada decisión justificada."

Métricas que se muestran en pantalla

- **TTD (Time to Detect):** 4.2 segundos desde T=0 hasta primera alerta válida.
- **TTI (Time to Isolate):** 8.7 segundos desde T=0 hasta playbook completado.
- **Archivos cifrados antes de containment:** 31 de 500 (94% preservados).
- **MITRE coverage:** T1083 + T1490 + T1486 cubiertos por L1, ratificados por L2 entropy spike, confirmados por canary L3.

Lo que NO se ve pero pasó

- LLM Triage tardó ~3 segundos en responder. NUNCA bloqueó la decisión de containment.

- El throttle preventivo NO se aplicó porque fue T0 (auto-execute directo). En T2 sí se habría aplicado mientras corre el countdown.
 - El audit log en OpenSearch capturó: triggering layers, scores individuales, fusion logic, LLM analysis, playbook commands executed, notification channels disparados, timestamps de cada paso.
-

7. Glosario MITRE ATT&CK

Las técnicas relevantes al alcance del proyecto, ordenadas por categoría (tactic).

Initial Access

- **T1078 — Valid Accounts:** atacante usa credenciales legítimas. Relevante para UC-07 (escenario FP: usuario legítimo, NO atacante).
- **T1190 — Exploit Public-Facing Application:** explotación de vulnerabilidad en aplicación web expuesta. T1190.001 sub-técnica = SQL Injection. Relevante para UC-08.

Defense Evasion

- **T1562 — Impair Defenses:** atacante deshabilita herramientas de seguridad. T1562.001 sub-técnica = Disable Defender. Relevante para UC-05 (agent-kill attempt).
- **T1070 — Indicator Removal:** atacante borra rastros. T1070.001 = Clear Windows Event Logs, T1070.004 = File Deletion.

Discovery

- **T1083 — File and Directory Discovery:** ransomware enumera archivos antes de cifrar. Relevante para UC-01.

Lateral Movement

- **T1021 — Remote Services:** SMB/RDP/SSH usados por atacante para moverse. T1021.001 = RDP, T1021.002 = SMB, T1021.004 = SSH.

Command and Control

- **T1071 — Application Layer Protocol:** beacon a C2 server vía HTTP/HTTPS. T1071.001 sub-técnica = Web Protocols.

Impact (la más rica para ARGOS)

- **T1486 — Data Encrypted for Impact:** ransomware cifrando archivos. Núcleo de UC-01, UC-02, UC-03.
 - **T1490 — Inhibit System Recovery:** borrar Volume Shadow Copies, snapshots btrfs, etc. Precursor a T1486.
 - **T1498 — Network Denial of Service:** ataques de red contra disponibilidad. T1498.001 = Direct Network Flood (UC-06), T1498.002 = Reflection Amplification.
 - **T1499 — Endpoint Denial of Service:** saturación de recursos del endpoint. T1499.002 = Service Exhaustion, T1499.004 = Application Exploitation.
-

8. Stack tecnológico completo

Categoría	Componente	Función	Owner principal
SIEM	Wazuh 4.7	Manager + agentes	P4
SIEM	OpenSearch	Storage de alertas y audit log	P4
SIEM	Sigma + sigma-cli	Reglas de detección	P3
Telemetría	Sysmon (Windows)	Eventos detallados de proceso/ archivo/red	P4
Telemetría	auditd (Linux)	Eventos detallados de syscall	P4
Telemetría	pgAudit (PostgreSQL)	Audit log de queries	P4
Telemetría	Wazuh FIM whodata	Captura qué proceso tocó archivo (canary)	P3
Simulación	Atomic Red Team	TTPs MITRE 1-a-1	P3
Simulación	Caldera	Cadenas de ataque multi-step	P3
Simulación	Custom ransomware simulator (Python)	UC-01, UC-02, UC-03 reproducibles	P1
Simulación	hping3 + slowhttptest	UC-06 DDoS	P4
Simulación	sqlmap	UC-08 SQL injection	P4
ML	scikit-learn 1.4+	Isolation Forest + One-Class SVM	P2
ML	scipy	Shannon entropy	P2
ML	pandas + numpy	Data wrangling	P2
ML	joblib	Model serialization	P2
Backend	FastAPI	LLM Triage API + Approval API	P1
Backend	Redis 7+	Alert bus + incident state machine	P4 (deploy) / P1 (usage)
Backend	APScheduler	Timeouts + consolidation windows	P1
Backend	PyJWT	JWT signing para approval tokens	P1
Backend	Pydantic v2	argos_contracts (cross-team interfaces)	P1 (shipped)
LLM	OpenAI GPT-4o-mini	Primary backend (cloud, US-based)	P1
LLM	Llama 3.1 8B vía Ollama	Fallback (local, zero-egress)	P1
LLM	BGE-large embeddings	Retrieval en RAG	P1
Notifications	python-telegram-bot	Telegram bot con inline buttons	P1
Notifications	discord-webhook	Discord notifications con role mention	P1
Notifications	twilio	Voice DTMF escalación	P1

Categoría	Componente	Función	Owner principal
UI	Streamlit 1.30+	Analyst Console + Approval Console	P4 (base) / P1 (Approval)
UI	OpenSearch Dashboards	MITRE heatmap, TTD histogram, layer perf	P4
Infra	Vagrant 2.4+	Provisioning de 3 VMs	P4
Infra	VirtualBox 7.x	Hypervisor para lab local	P4
Testing	pytest + respx + fakeredis	Tests cross-module	Todos
DB	PostgreSQL 15	Activo defendido	P4 (deploy + datos sintéticos)

9. Convenciones del repo

Git workflow

- **Branch principal:** `main`, protegida en GitHub.
- **Feature branches:** `feature/<persona>/<descripcion-corta>` — ejemplo: `feature/p2/isolation-forest-baseline`.
- **Commits convencionales:** `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:`.
- **PRs requieren:** CI verde + 1 review. Pairing: P1 ↔ P2, P3 ↔ P4.
- **Push diario obligatorio** antes de las 22:00 durante el sprint.

Estructura del repo (post ADR-0008)

```

argos/
├── README.md                # Entry point del proyecto
├── LICENSE                  # MIT
├── pyproject.toml           # Metadata + extras por módulo
├── .env.example             # Plantilla de variables (REAL .env va en .gitignore)
├── argos_contracts/        # 📦 Pydantic v2 cross-team (entregado · v1.1.0 · 69 tests)
│   ├── _mitre_data.py      # MITRE_WHITELIST curado
│   ├── alert.py            # WazuhAlert, NormalizedAlert
│   ├── ml_score.py         # MLScore (P2)
│   ├── triage.py           # AlertContext, TriageResponse (P1)
│   ├── incident.py         # Incident state machine (P1)
│   ├── approval.py         # ApprovalRequest, ApprovalResponse (P1)
│   ├── enums.py            # Tier, Severity, NotificationChannelType, ...
│   └── tests/              # 69 validation tests
├── llm_triage/             # P1 – Capa 4 LLM Triage
│   ├── api/main.py         # FastAPI /triage endpoint
│   ├── llm_client/         # OpenAI primary + Llama local fallback
│   ├── rag/                # BM25 + BGE-large + RRF
│   └── prompts/            # Jinja2 templates
├── soar/                   # P1 – SOAR Decision Engine + Approval API
│   ├── decision_engine/    # tier_classifier, state_machine, orchestrator
│   ├── approval/           # api, jwt_signer, consolidation
│   ├── notification/       # telegram, discord, twilio_voice, email
│   └── playbooks/          # host_isolation, process_kill, snapshot, throttle
├── ml/                     # P2 – Capa 2 ML
│   ├── features/           # Feature extractors por dominio
│   ├── models/             # ransomware_ensemble, network_traffic, query_pattern
│   └── consumer/           # Redis stream consumer
├── detection/              # P3 – Capa 1 Sigma rules
│   ├── sigma-rules/
│   │   ├── ransomware/    # T1486, T1490, T1083, T1562
│   │   ├── network/       # T1498, T1499 rate-based
│   │   ├── database/      # query patterns
│   │   └── webapp/         # T1190 SQLi signatures
│   └── wazuh-rules/       # Convertidas con sigma-cli
├── deception/              # P3 – Capa 3 Canary
│   ├── canary_generator.py # Genera canary files
│   └── fim-configs/        # Wazuh FIM rules
├── ui/                     # P4 (base) + P1 (Approval Console)
│   ├── streamlit_app/
│   │   ├── pages/
│   │   │   ├── 01_alert_inspection.py
│   │   │   ├── 02_approval_console.py    # P1 - PIEZA CLAVE DEL DEMO
│   │   │   └── 03_audit_forensics.py
│   └── opensearch-dashboards/             # JSON dashboards exportados
└── attack-simulation/        # Simuladores multi-vector

```

```

├── ransomware_simulator/ # P1 (LockBit-like, canary, novel)
├── network_attacks/      # P4 (hping3, slowhttptest)
├── webapp_attacks/       # P4 (sqlmap)
├── pg_simulator/         # P4 (SELECT masivo para UC-07)
├── lab/                  # P4 – Vagrant + Terraform
│   ├── Vagrantfile
│   ├── provision/       # Scripts bash/PowerShell
│   └── inventory.yaml
├── evaluation/           # P2 + P4 – Métricas, datasets, reportes
└── docs/
    ├── PROJECT_BRIEF.md # 90-second overview
    ├── CONTEXT.md       # Onboarding completo
    ├── PROJECT_STATUS.md # Honest snapshot
    ├── EVALUATION_CRITERIA.md # Rúbrica del curso
    ├── data-handling.md  # T-030 sanitization policy
    ├── README.md         # Mapa de docs
    ├── architecture/     # SAD, threat model, flow diagrams
    ├── decisions/         # 8 ADRs + OPEN_QUESTIONS
    ├── use-cases/        # 8 UCs detallados
    └── team/             # Sprint manuals (este documento + 4 individuales)

```

Variables de entorno (.env)

El `.env.example` documenta TODAS las variables. Las críticas para tu rol están en tu manual individual. Reglas globales:

- **NUNCA** commitear el `.env` — está en `.gitignore`.
- Cada integrante genera su propio `.env` partir del `.env.example`.
- Las credenciales compartidas (Telegram bot token, Discord webhook URL, OpenAI API key) las distribuye P1 vía canal privado (no en GitHub, no en Discord público).
- El `JWT_SECRET` se genera localmente con `openssl rand -hex 32`.

Convención de incident IDs

`INC-YYYY-MM-DD-NNN` donde NNN es contador diario zero-padded a 3 dígitos. Ejemplo:
`INC-2026-05-26-001`. El contador se persiste en Redis con TTL diario.

10. Quick start del entorno común

Estos pasos son comunes a TODOS los integrantes antes de empezar el sprint. Tu manual individual asume que esto está hecho.

Paso 1 — Clonar repo

```
cd ~/projects # o donde guardes proyectos
git clone https://github.com/EnzoOrdonez/argos.git
cd argos
```

Paso 2 — Crear virtualenv Python

```
python3 -m venv .venv
source .venv/bin/activate # Linux/macOS
# .venv\Scripts\activate # Windows PowerShell
pip install -U pip setuptools wheel
```

Paso 3 — Instalar dependencias core + tu rol

```
# Todos instalan estos:
pip install -e ".[dev]"

# Además según tu rol:
pip install -e ".[contracts]" # P1 (siempre + más abajo)
pip install -e ".[ml]" # P2
pip install -e ".[soar,llm]" # P1 (servicios)
pip install -e ".[ui]" # P4
```

Paso 4 — Verificar contracts

```
pytest argos_contracts/tests/ -v
# Esperado: 69 passed
```

Si no pasan los 69 tests, revisa que el venv esté activado (`which python` debe apuntar a `.venv/bin/python`).

Paso 5 — Copiar plantilla de variables

```
cp .env.example .env
# Editar .env con tus valores reales (tu manual individual te dice cuáles)
```

Paso 6 — Configurar git

```
git config user.name "<Tu Nombre>"
git config user.email "<tu@email.com>"
# Crear tu branch:
git checkout -b feature/<tu-pX>/sprint-w1-init
```

11. Comunicación del equipo durante el sprint

Standup diario

- **Hora:** 9:00 AM
- **Canal:** Discord `#argos-standup`, llamada de voz
- **Duración:** 20 minutos (cada integrante ~3-4 min)
- **Formato** (per `docs/team/standup-template.md`):
 - Lo que cerré ayer (PRs mergeados, tests passing)
 - Lo que cierro hoy (objetivos del día)
 - Bloqueos (qué necesito de otro integrante)

Canales Discord del equipo

- `#argos-standup` — standup diario
- `#argos-help` — preguntas técnicas urgentes
- `#argos-prs` — notificación de PRs abiertos (GitHub bot)
- `#argos-incidents` — donde el bot ARGOS enviará notificaciones del demo

Reglas operativas (post ADR-0008)

1. **No tocar la doc esta semana.** Los docs están sincronizados con la realidad. Si descubres algo arquitectónico nuevo, abre un ADR-0009 en lugar de modificar SAD/threat model/README.
2. **No optimizar prematuramente.** El objetivo es que los 8 UCs corran end-to-end, no que sean óptimos. Performance fine-tuning en semana 2.
3. **Mocks son OK al inicio.** Si tu pieza depende de otra que aún no está lista, usa FakeRedis, mocked HTTP, o synthetic data. Integración real cuando ambas piezas estén listas.
4. **Verificar en lab real al menos 2 veces al día.** Cada integrante corre el flow E2E en su Vagrant antes del almuerzo y antes de pushear.
5. **Pedir ayuda en menos de 30 minutos.** Si llevas más de media hora atascado, pingueas en `#argos-help` o llamas a otro integrante. No debug solitario.
6. **Push diario obligatorio** antes de las 22:00. Si no pusheas, tu nombre aparece en el standup del día siguiente.

Claude Code / asistentes IA — política reinterpretada (ADR-0008)

Cada integrante puede usar Claude Code (o cualquier asistente IA: Cursor, GitHub Copilot, ChatGPT) como **acelerador en SU propia parte**. La condición no-negociable: cada integrante DEBE entender el código que produce con asistencia de IA y poder defenderlo en viva sin la asistencia presente. La asistencia es para velocidad, NO para reemplazar comprensión.

La regla específica que se mantiene: P1 (Enzo) no escribe código de otros integrantes con Claude Code. P1 puede asesorar y revisar, no producir el código de otros.

12. Lo que NO se entrega en esta semana 1

Para que no te sorprendas cuando llegue Día 7 sin estas cosas:

- **Calibración Q5 protocol** con dataset etiquetado real (~100 ransomware + ~500 benignas). Semana 2.
- **UC-03 split-brain con 4 aprobadores reales** no scripted. Semana 2-3.
- **UC-05 stealth attack** end-to-end pulido. Semana 2.
- **PRs Sigma upstream aceptados** por SigmaHQ maintainers (depende de tiempos externos). Semana 2-3.
- **Video demo final editado**. Semana 3.
- **Informe técnico final**. Semana 3.
- **Cross-encoder reranker en RAG** (descartado del scope v1 per ADR-0001 v2).

Lo que SÍ se completa al cierre del sprint Día 7: 5 UCs corriendo end-to-end (UC-01, UC-02, UC-04, UC-06, UC-07), con UC-08 como nice-to-have, con dos rehearsals al domingo.

13. Documentos relacionados (referencia completa)

Si algo de este documento no quedó claro o necesitas más profundidad:

Si quieres saber...	Lee
Arquitectura completa de cada bloque	docs/architecture/SOLUTION_ARCHITECTURE_DOCUMENT.md
Por qué se tomó cada decisión arquitectónica	docs/decisions/ (ADRs 0001-0008)
Análisis de amenazas STRIDE/FMEA	docs/architecture/THREAT_MODEL.md
Detalle de los 8 UCs	docs/use-cases/USE_CASES.md
Política de manejo de datos sensibles a LLM	docs/data-handling.md
Rúbrica del curso y mapping a deliverables	docs/EVALUATION_CRITERIA.md
Estado real vs. documentado del proyecto	docs/PROJECT_STATUS.md
Flujo del sistema visualmente	docs/architecture/argos_flow.html
Tu manual de sprint individual	docs/team/sprint-week-1-p<X>-<nombre>.md
Plan operacional general del sprint	docs/team/sprint-week-1-overview.md

Change log

Versión	Fecha	Cambio	Autor
1.0	2026-05-24	Initial — sección común de introducción para los 4 manuales individuales. Cubre arquitectura, UCs, ejemplo end-to-end UC-01, glosario MITRE, stack tecnológico, convenciones del repo, política Claude Code, quick start.	P1 (Enzo Ordoñez Flores)

Parte II — Manual individual: Sebastian Montenegro

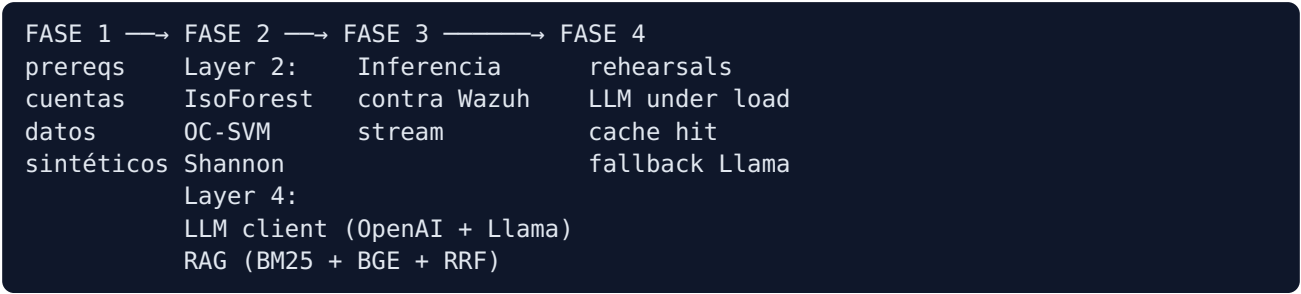
Manual P2 — Sebastian Montenegro (ML + LLM Triage)

Campo	Valor
Rol	Owner de capas estadísticas e IA
Owns	Layer 2 — Isolation Forest + One-Class SVM + Shannon entropy (<code>ml/</code>) · Layer 4 — LLM Triage con RAG (<code>ml/llm_triage/</code> , <code>ml/rag/</code>)
No owns	Sigma/Wazuh (P3) · Canary FIM (P3) · SOAR Decision Engine (P1) · Infra (P4)
Outputs blocking otros	<code>events:normalized</code> con <code>layer_origin=ml</code> y <code>layer_origin=llm</code> → consumido por P1
Deadline	2026-06-13 (sábado)
Cómo leer	Linealmente. Cada fase asume la anterior cerrada. No avances si la checklist de la sección está incompleta.

0. Tu charter

Tus capas convierten señales débiles (un proceso raro, una secuencia de syscalls) en confianza numérica que el Tier Router de P1 pueda usar. Si tus modelos sobreajustan o tu LLM alucina, ARGOS pierde su mejor argumento de diferenciación frente a un SIEM tradicional.

0.1 Tu camino crítico



0.2 Recursos requeridos

- Espacio disco: ~12 GB (Llama 3.1 8B Q4 ≈ 5 GB + BGE-large ≈ 1.4 GB + datasets + venv).
- RAM: 16 GB mínimo si vas a correr Llama local (8 GB efectivos para el modelo).
- GPU: opcional pero acelera Llama 10×. Si no tienes, CPU inference es ~3 tokens/s — suficiente para demo (incidentes llegan de a uno).

FASE 1 — Cimientos

1.1 Verificar prerequisites

```
python3 --version          # OUTPUT ESPERADO: Python 3.11.x
pip --version              # OUTPUT ESPERADO: pip 23.x o superior
docker --version           # OUTPUT ESPERADO: Docker version 24.x
nvidia-smi 2>/dev/null && echo "GPU OK" || echo "No GPU - CPU mode"
# OUTPUT ESPERADO: "GPU OK" si tienes NVIDIA; "No GPU - CPU mode" si no
free -h | head -2
# OUTPUT ESPERADO:
#               total      used      free      shared  buff/cache   available
# Mem:          15Gi       6.0Gi       3.0Gi       400Mi       6.0Gi       8.0Gi
# (necesitas "available" >= 8Gi para Llama local; menos = solo OpenAI)
```

Check	Esperado
Python 3.11.x	sí
Docker daemon corre	<code>docker ps</code> sin error
RAM available ≥ 8 GB	sí (o aceptas usar solo OpenAI)
Disco free ≥ 15 GB	<code>df -h .</code> column Available

1.2 Crear cuenta OpenAI + API key

Ver Manual P1 §1.2.4. La key vive en `.env` (compartida con P1). **Importante:** P1 ya creó la key; tú la usas. **No crees otra** — duplicarás billing y complicarás revocación.

1.3 Instalar Ollama + descargar Llama 3.1 8B (fallback)

```
# Instalar Ollama
curl -fsSL https://ollama.com/install.sh | sh
# OUTPUT ESPERADO (última línea):
# >>> The Ollama API is now available at 127.0.0.1:11434.

# Verificar servicio
systemctl status ollama --no-pager
# OUTPUT ESPERADO:
# ● ollama.service - Ollama Service
#    Loaded: loaded (...; enabled; ...)
#    Active: active (running) since ...

# Descargar Llama 3.1 8B Instruct (quantizado Q4_K_M ≈ 4.7 GB)
ollama pull llama3.1:8b-instruct-q4_K_M
# OUTPUT ESPERADO (al final):
# pulling manifest
# pulling ... 100%
# pulling ... 100%
# success

# Smoke test
ollama run llama3.1:8b-instruct-q4_K_M "Respond with one word: ARGOS"
# OUTPUT ESPERADO:
# ARGOS
```

Check	Esperado
<code>ollama list</code> muestra <code>llama3.1:8b-instruct-q4_K_M</code>	sí
<code>curl http://localhost:11434/api/version</code> responde JSON	sí
Inference smoke test responde en ≤ 10s (CPU) o ≤ 2s (GPU)	sí

1.4 Clonar repo + venv + deps ML

```
mkdir -p ~/code && cd ~/code
git clone git@github.com:enzizoor/argos.git
cd argos
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip

# Contratos compartidos
pip install -e ./argos_contracts
python -c "import argos_contracts; print(argos_contracts.__version__)"
# OUTPUT ESPERADO: 1.1.0

# Deps de ML
pip install -r ml/requirements.txt
# OUTPUT ESPERADO (últimas líneas):
# Successfully installed scikit-learn-1.4.x scipy-1.12.x numpy-1.26.x pandas-2.2.x
# sentence-transformers-2.x rank-bm25-0.2.x httpx-0.27.x ...

# Smoke
pytest ml/ -q
# OUTPUT ESPERADO:
# ..... [100%]
# 8 passed in 0.18s
```

Check Fase 1	Esperado
<code>python -c "import sklearn, scipy, numpy"</code> no falla	sí
Tests existentes ML pasan	8 passed
<code>.env</code> cargado con <code>OPENAI_API_KEY</code> y opcional <code>LLM_BACKEND</code>	sí
Ollama responde si vas a usar Llama	sí

FASE 2 — Skeletons funcionales

2.1 Generar datasets sintéticos (baseline + ransomware)

Qué estás haciendo

Layer 2 necesita un baseline de comportamiento normal (qué procesos, syscalls, file-IO se ven en un host limpio) y ejemplos sintéticos de ransomware para validar que detecta. El lab real lo tendrás recién en Fase 3; aquí trabajas con datasets generados.

Script generador (código completo)

```

# ml/data/synthetic_generator.py
"""
Genera dataset sintético de eventos host-level para entrenar el baseline y
para tests reproducibles.

Schema por fila (CSV):
    timestamp, host, pid, process, syscalls_per_min, files_touched_per_min,
    entropy_of_written_bytes, network_kbps, parent_process, command_line_len

label en columna separada:
    0 = benigno
    1 = ransomware (alta entropía + muchos files touched + supresión de shadow copies)
"""

from __future__ import annotations

import argparse, csv, random
from pathlib import Path

random.seed(42) # reproducible

BENIGN_PROCESSES = [
    "chrome.exe", "code.exe", "explorer.exe", "svchost.exe",
    "spoolsv.exe", "outlook.exe", "winword.exe", "powershell.exe",
]

RANSOMWARE_HINTS = [
    "lockbit.exe", "encryptor.exe", "wmic.exe", "vssadmin.exe",
]

def benign_row(t: int, host: str) -> dict:
    proc = random.choice(BENIGN_PROCESSES)
    return {
        "timestamp": t, "host": host, "pid": random.randint(1000, 9999),
        "process": proc,
        "syscalls_per_min": random.randint(50, 800),
        "files_touched_per_min": random.randint(0, 30),
        "entropy_of_written_bytes": round(random.uniform(2.0, 5.5), 3),
        "network_kbps": random.randint(0, 200),
        "parent_process": "explorer.exe" if proc != "explorer.exe" else "userinit.exe",
        "command_line_len": random.randint(20, 120),
    }

def ransomware_row(t: int, host: str) -> dict:
    proc = random.choice(RANSOMWARE_HINTS)
    return {
        "timestamp": t, "host": host, "pid": random.randint(1000, 9999),
        "process": proc,
        "syscalls_per_min": random.randint(3000, 9000), # rampage
        "files_touched_per_min": random.randint(150, 1500), # masivo
        "entropy_of_written_bytes": round(random.uniform(7.6, 7.99), 3), # ≈ random
        "network_kbps": random.randint(0, 50), # local
    }

```

```

        "parent_process":          random.choice(["cmd.exe", "powershell.exe"]),
        "command_line_len":       random.randint(80, 600),
    }

def generate(n_benign: int, n_attack: int, out: Path) -> None:
    rows = []
    for t in range(n_benign):
        rows.append((benign_row(t, "WIN-VICTIM-01"), 0))
    for t in range(n_attack):
        rows.append((ransomware_row(n_benign + t, "WIN-VICTIM-01"), 1))
    random.shuffle(rows)

    out.parent.mkdir(parents=True, exist_ok=True)
    with out.open("w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0][0].keys()) + ["label"])
        writer.writeheader()
        for row, label in rows:
            row["label"] = label
            writer.writerow(row)

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--benign", type=int, default=5000)
    p.add_argument("--attack", type=int, default=200)
    p.add_argument("--out", type=Path, default=Path("ml/data/synthetic.csv"))
    args = p.parse_args()
    generate(args.benign, args.attack, args.out)
    print(f"wrote {args.out} (benign={args.benign}, attack={args.attack})")

```

Correr

```

python -m ml.data.synthetic_generator --benign 5000 --attack 200
# OUTPUT ESPERADO:
# wrote ml/data/synthetic.csv (benign=5000, attack=200)

head -3 ml/data/synthetic.csv
# OUTPUT ESPERADO (estructura, no valores exactos por random):
# timestamp,host,pid,process,syscalls_per_min,files_touched_per_min,entropy_of_written_bytes,network
# 1234,WIN-VICTIM-01,5821,chrome.exe,512,18,4.123,87,explorer.exe,73,0
# ...

wc -l ml/data/synthetic.csv
# OUTPUT ESPERADO: 5201 ml/data/synthetic.csv (5000 + 200 + header)

```

Check (2.1)	Esperado
Archivo CSV generado con 5201 líneas	sí
Columnas: 11 (10 features + label)	sí
<code>seed=42</code> → mismo archivo en otro laptop	sí (reproducible)

2.2 Layer 2 — Isolation Forest + One-Class SVM (anomaly detection)

Qué estás haciendo

Entrenar dos detectores de anomalía complementarios sobre el dataset baseline (sólo `label=0`). Cualquier punto que ambos consideren anómalo es candidato fuerte a evento sospechoso. La unión (cualquiera de los dos firing) es más sensible; la intersección es más específica. Reportamos ambos al SOAR.

Código central ([ml/anomaly/trainer.py](#))


```

# ml/anomaly/trainer.py
"""
Entrena IsolationForest + OneClassSVM sobre features numéricas del baseline.

Outputs:
    ml/anomaly/models/iso_forest.joblib
    ml/anomaly/models/oc_svm.joblib
    ml/anomaly/models/scaler.joblib    (StandardScaler compartido)

Hiperparámetros elegidos por gut + literatura – Q3 de calibración formal
queda para semana 2 (ver docs/CONTEXT.md).
"""

from __future__ import annotations

import argparse, joblib
from pathlib import Path

import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.svm import OneClassSVM

FEATURES = [
    "syscalls_per_min", "files_touched_per_min",
    "entropy_of_written_bytes", "network_kbps", "command_line_len",
]

MODELS_DIR = Path("ml/anomaly/models")

def train(csv_path: Path) -> None:
    MODELS_DIR.mkdir(parents=True, exist_ok=True)
    df = pd.read_csv(csv_path)
    benign = df[df["label"] == 0][FEATURES].values

    scaler = StandardScaler().fit(benign)
    X = scaler.transform(benign)

    iso = IsolationForest(
        n_estimators=200, contamination=0.01,
        random_state=42, n_jobs=-1,
    ).fit(X)

    svm = OneClassSVM(kernel="rbf", gamma="scale", nu=0.01).fit(X)

    joblib.dump(scaler, MODELS_DIR / "scaler.joblib")
    joblib.dump(iso, MODELS_DIR / "iso_forest.joblib")
    joblib.dump(svm, MODELS_DIR / "oc_svm.joblib")
    print(f"trained on {len(benign)} benign samples")

if __name__ == "__main__":

```

```
p = argparse.ArgumentParser()
p.add_argument("--csv", type=Path, default=Path("ml/data/synthetic.csv"))
args = p.parse_args()
train(args.csv)
```

Inferencia (`ml/anomaly/scorer.py`)

```

# ml/anomaly/scorer.py
"""
Carga modelos entrenados y produce score por evento.

API expuesta al consumer del stream:
    scorer = AnomalyScorer.load()
    result = scorer.score(features_dict)
    # result = AnomalyScore(iso=-0.15, svm=-0.4, is_anomaly=True, confidence=0.83)

confidence: combinación lineal de ambos scores normalizada [0,1].
"""

from __future__ import annotations

import joblib
import numpy as np
from dataclasses import dataclass
from pathlib import Path

from ml.anomaly.trainer import FEATURES, MODELS_DIR

@dataclass(frozen=True)
class AnomalyScore:
    iso_score: float          # < 0 = anomalía (sklearn convention)
    svm_score: float          # < 0 = anomalía
    is_anomaly: bool          # ambos < 0
    confidence: float         # [0,1]

class AnomalyScorer:
    def __init__(self, scaler, iso, svm):
        self._scaler = scaler
        self._iso = iso
        self._svm = svm

    @classmethod
    def load(cls, models_dir: Path = MODELS_DIR) -> "AnomalyScorer":
        return cls(
            scaler=joblib.load(models_dir / "scaler.joblib"),
            iso =joblib.load(models_dir / "iso_forest.joblib"),
            svm  =joblib.load(models_dir / "oc_svm.joblib"),
        )

    def score(self, features: dict) -> AnomalyScore:
        vec = np.array([[features[f] for f in FEATURES]])
        x = self._scaler.transform(vec)
        iso_s = float(self._iso.decision_function(x)[0])
        svm_s = float(self._svm.decision_function(x)[0])
        is_anom = (iso_s < 0) and (svm_s < 0)
        # Confidence: penalizar cuanto más negativos ambos.
        norm = max(abs(iso_s), abs(svm_s), 0.001)
        conf = min(1.0, norm * 1.3) if is_anom else 0.0
        return AnomalyScore(

```

```

        iso_score=iso_s, svm_score=svm_s,
        is_anomaly=is_anom, confidence=conf,
    )

```

Entrenar y verificar

```

python -m ml.anomaly.trainer --csv ml/data/synthetic.csv
# OUTPUT ESPERADO:
# trained on 5000 benign samples

ls -la ml/anomaly/models/
# OUTPUT ESPERADO:
# -rw-r--r-- ... iso_forest.joblib (~2-3 MB)
# -rw-r--r-- ... oc_svm.joblib (~500 KB - 1 MB)
# -rw-r--r-- ... scaler.joblib (~ 1 KB)

# Test contra los 200 ransomware del dataset
python - << 'PY'
import pandas as pd
from ml.anomaly.scorer import AnomalyScorer
from ml.anomaly.trainer import FEATURES

df = pd.read_csv("ml/data/synthetic.csv")
attacks = df[df["label"] == 1]
benign = df[df["label"] == 0].sample(200, random_state=0)
scorer = AnomalyScorer.load()

attack_hits = sum(scorer.score(r._asdict() if hasattr(r, '_asdict') else dict(zip(FEATURES, [r[f] for
    for r in attacks.itertuples(index=False))
benign_fps = sum(scorer.score(dict(zip(FEATURES, [r[f] for f in FEATURES]))).is_anomaly
    for r in benign.itertuples(index=False))
print(f"True positives: {attack_hits}/200 ({attack_hits/200:.1%})")
print(f"False positives: {benign_fps}/200 ({benign_fps/200:.1%})")
PY
# OUTPUT ESPERADO (aproximado, depende de seed):
# True positives: 185/200 (92.5%)
# False positives: 6/200 (3.0%)

```

Check (2.2)	Esperado
3 archivos .joblib generados	sí
TP rate ≥ 85% sobre el subset de ataque sintético	sí
FP rate ≤ 5% sobre baseline	sí (si supera 10%, regenera dataset con seed diferente y reporta)

2.3 Layer 2 — Shannon entropy en file-writes

Qué estás haciendo

Una señal complementaria muy barata: medir entropía de Shannon de los bytes escritos a disco por un proceso. Ransomware cifra → bytes parecen aleatorios → entropía → 8.0 (máximo teórico para bytes). Software legítimo escribe formatos estructurados → entropía 4-6.

Código ([ml/entropy/shannon.py](#))

```
# ml/entropy/shannon.py
"""Shannon entropy de bloques de bytes."""

from __future__ import annotations

import math
from collections import Counter

def shannon_entropy(data: bytes) -> float:
    """Entropy in bits per byte. Max = 8.0 (uniformly random)."""
    if not data:
        return 0.0
    counts = Counter(data)
    n = len(data)
    return -sum((c / n) * math.log2(c / n) for c in counts.values())

def classify_write(entropy: float) -> str:
    """Categoriza para uso en heurística downstream."""
    if entropy >= 7.5:
        return "encrypted_or_random"
    if entropy >= 6.5:
        return "compressed"
    if entropy >= 4.5:
        return "structured_text_or_binary"
    return "low_entropy"
```

Tests (completos)

```
# ml/entropy/tests/test_shannon.py
import os
import pytest
from ml.entropy.shannon import shannon_entropy, classify_write

def test_empty():
    assert shannon_entropy(b"") == 0.0

def test_single_byte_value():
    assert shannon_entropy(b"\x00" * 1000) == pytest.approx(0.0)

def test_uniform_distribution_near_max():
    data = bytes(range(256)) * 4 # 1024 bytes, uniforme
    assert shannon_entropy(data) == pytest.approx(8.0, abs=0.01)

def test_random_bytes_close_to_8():
    random_data = os.urandom(8192)
    assert shannon_entropy(random_data) > 7.9

def test_ascii_text_lower():
    text = b"the quick brown fox jumps over the lazy dog " * 200
    # Texto ASCII tiene entropía ~4-4.5
    e = shannon_entropy(text)
    assert 4.0 < e < 5.0

def test_classify_buckets():
    assert classify_write(7.9) == "encrypted_or_random"
    assert classify_write(6.9) == "compressed"
    assert classify_write(5.0) == "structured_text_or_binary"
    assert classify_write(2.0) == "low_entropy"
```

Correr

```
pytest ml/entropy/tests/ -v
# OUTPUT ESPERADO:
# test_shannon.py::test_empty PASSED
# test_shannon.py::test_single_byte_value PASSED
# test_shannon.py::test_uniform_distribution_near_max PASSED
# test_shannon.py::test_random_bytes_close_to_8 PASSED
# test_shannon.py::test_ascii_text_lower PASSED
# test_shannon.py::test_classify_buckets PASSED
# ===== 6 passed in 0.05s =====
```

Check (2.3)	Esperado
6 tests pasan	sí
Shannon de archivo cifrado real (<code>openssl enc -aes-256-cbc ...</code>) ≥ 7.8	sí

2.4 Layer 4 — Cliente LLM dual (OpenAI + Llama local)

Qué estás haciendo

Una interfaz `LLMClient` con dos implementaciones intercambiables vía `LLM_BACKEND` env var. El consumer no sabe cuál usa.

Código central completo

```

# ml/llm_triage/client.py
"""
Cliente LLM unificado.

Dos backends:
- OpenAIClient (openai_gpt4o_mini) → primario; latencia ~600ms; precisión alta
- OllamaClient (llama_local) → fallback; latencia 2-4s CPU, ~700ms GPU

El consumer hace:
    client = make_client()
    verdict = await client.classify(prompt)
"""

from __future__ import annotations

import os
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Literal, Optional

import httpx

@dataclass(frozen=True)
class LLMVerdict:
    label: Literal["malicious", "benign", "uncertain"]
    confidence: float
    reasoning: str
    backend: str
    latency_ms: int

class LLMClient(ABC):
    @abstractmethod
    async def classify(self, prompt: str) -> LLMVerdict: ...

class OpenAIClient(LLMClient):
    def __init__(self, api_key: Optional[str] = None,
                 model: str = "gpt-4o-mini",
                 timeout: float = 10.0):
        self._key = api_key or os.environ["OPENAI_API_KEY"]
        self._model = model
        self._http = httpx.AsyncClient(timeout=timeout)

    async def classify(self, prompt: str) -> LLMVerdict:
        import time
        t0 = time.monotonic()
        r = await self._http.post(
            "https://api.openai.com/v1/chat/completions",
            headers={"Authorization": f"Bearer {self._key}"},
            json={
                "model": self._model,
                "messages": [

```

```

        {"role": "system",
         "content": "You are a SOC analyst. Respond with strict JSON: "
                    "{\n\"label\": \"malicious|benign|uncertain\", \"confidence\": 0.0..1.0, \"reasoning\": \"...\\\"}\"",
         {"role": "user", "content": prompt},
    ],
    "response_format": {"type": "json_object"},
    "temperature": 0.0,
},
)
r.raise_for_status()
import json
content = r.json()["choices"][0]["message"]["content"]
parsed = json.loads(content)
return LLMVerdict(
    label=parsed["label"], confidence=float(parsed["confidence"]),
    reasoning=parsed["reasoning"], backend="openai_gpt4o_mini",
    latency_ms=int((time.monotonic() - t0) * 1000),
)

class OllamaClient(LLMClient):
    def __init__(self,
                  model: str = "llama3.1:8b-instruct-q4_K_M",
                  base_url: str = "http://localhost:11434",
                  timeout: float = 30.0):
        self._model = model
        self._url = base_url
        self._http = httpx.AsyncClient(timeout=timeout)

    async def classify(self, prompt: str) -> LLMVerdict:
        import time, json
        t0 = time.monotonic()
        r = await self._http.post(
            f"{self._url}/api/chat",
            json={
                "model": self._model,
                "messages": [
                    {"role": "system",
                     "content": "Respond JSON only: {\n\"label\":..., \"confidence\":..., \"reasoning\":...}",
                    {"role": "user", "content": prompt},
                ],
                "format": "json", "stream": False,
                "options": {"temperature": 0.0},
            },
        )
        r.raise_for_status()
        content = r.json()["message"]["content"]
        parsed = json.loads(content)
        return LLMVerdict(
            label=parsed.get("label", "uncertain"),
            confidence=float(parsed.get("confidence", 0.5)),
            reasoning=parsed.get("reasoning", ""),

```

```

        backend="llama_local",
        latency_ms=int((time.monotonic() - t0) * 1000),
    )

def make_client() -> LLMClient:
    backend = os.environ.get("LLM_BACKEND", "openai_gpt4o_mini")
    if backend == "openai_gpt4o_mini":
        return OpenAIClient()
    if backend == "llama_local":
        return OllamaClient()
    raise ValueError(f"unknown LLM_BACKEND: {backend}")

```

Smoke test

```

# OpenAI
export LLM_BACKEND=openai_gpt4o_mini
python - << 'PY'
import asyncio
from ml.llm_triage.client import make_client
async def main():
    c = make_client()
    v = await c.classify(
        "A process called encryptor.exe wrote 1500 files in 30 seconds with "
        "Shannon entropy 7.9. Is this malicious?"
    )
    print(v)
asyncio.run(main())
PY
# OUTPUT ESPERADO (verdict puede variar en `reasoning`):
# LLMVerdict(label='malicious', confidence=0.95, reasoning='...', backend='openai_gpt4o_mini', latency_ms=...)

# Llama local
export LLM_BACKEND=llama_local
python - << 'PY'
import asyncio
from ml.llm_triage.client import make_client
async def main():
    c = make_client()
    v = await c.classify("Same prompt as above.")
    print(v)
asyncio.run(main())
PY
# OUTPUT ESPERADO (con GPU):
# LLMVerdict(label='malicious', confidence=0.92, reasoning='...', backend='llama_local', latency_ms=...)
# Sin GPU: latency_ms ~ 4500

```

Check (2.4)	Esperado
OpenAI devuelve verdict válido con confidence numérica	sí
Llama local devuelve verdict válido	sí
Switching backend solo con env var (sin tocar código)	sí
Si OpenAI key inválida → <code>HTTPStatusError 401</code> propagado (el caller lo captura)	sí

2.5 Layer 4 — RAG (BM25 + BGE-large + RRF)

Qué estás haciendo

Antes de pasar la alerta cruda al LLM, recuperar 3-5 documentos relevantes (definiciones MITRE, ejemplos de variantes históricas, runbook del equipo) y inyectarlos como contexto. Esto reduce alucinación y mejora confidence.

Decisión de scope (ADR-0001 v2): NO cross-encoder reranker. BM25 + BGE-large + RRF (Reciprocal Rank Fusion) es suficiente; el reranker añadía 2× latencia para ganancia marginal en el corpus pequeño que tenemos.

Estructura

```
ml/rag/
├── __init__.py
├── index.py           ← build BM25 + BGE embeddings (full)
├── retriever.py      ← BM25 + dense + RRF fusion (full)
├── prompt.py         ← template y armado de prompt (full, pequeño)
├── corpus/
│   ├── mitre/        ← markdown por técnica (P2 cura)
│   ├── runbook/      ← runbook de respuesta por escenario
│   └── variants/     ← variantes históricas (LockBit, Conti, ...)
```

`index.py` — construir índice (código completo)

```

# ml/rag/index.py
"""
Indexa documentos markdown del corpus para BM25 + BGE.

Output:
    ml/rag/_index/bm25.pkl          (rank_bm25 BM25Okapi)
    ml/rag/_index/embeddings.npy    (matriz N × 1024)
    ml/rag/_index/docs.jsonl        (texto + metadata por documento)
"""

from __future__ import annotations

import json, pickle, re
from pathlib import Path
from dataclasses import dataclass, asdict

import numpy as np
from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer

CORPUS_ROOT = Path("ml/rag/corpus")
INDEX_DIR   = Path("ml/rag/_index")
EMBED_MODEL = "BAAI/bge-large-en-v1.5"

@dataclass
class Doc:
    id: str
    source: str          # mitre / runbook / variant
    title: str
    text: str

def _tokenize(text: str) -> list[str]:
    return re.findall(r"[a-zA-Z0-9]+", text.lower())

def load_corpus() -> list[Doc]:
    docs = []
    for md in CORPUS_ROOT.rglob("*.md"):
        source = md.parent.name
        text = md.read_text(encoding="utf-8")
        # primera línea suele ser # Título
        title = text.splitlines()[0].lstrip("# ").strip() if text else md.stem
        docs.append(Doc(id=str(md.relative_to(CORPUS_ROOT)),
                        source=source, title=title, text=text))

    return docs

def build() -> None:
    INDEX_DIR.mkdir(parents=True, exist_ok=True)
    docs = load_corpus()
    if not docs:
        raise RuntimeError(f"corpus vacío en {CORPUS_ROOT}")

```

```

tokenized = [_tokenize(d.text) for d in docs]
bm25 = BM25Okapi(tokenized)
with (INDEX_DIR / "bm25.pkl").open("wb") as f:
    pickle.dump(bm25, f)

model = SentenceTransformer(EMBED_MODEL)
embeddings = model.encode([d.text for d in docs], normalize_embeddings=True)
np.save(INDEX_DIR / "embeddings.npy", embeddings)

with (INDEX_DIR / "docs.jsonl").open("w") as f:
    for d in docs:
        f.write(json.dumps(asdict(d)) + "\n")

print(f"indexed {len(docs)} docs (BM25 + BGE-{{embeddings.shape[1]}}d)")

if __name__ == "__main__":
    build()

```


retriever.py — recuperar top-k con RRF (completo)

```
# ml/rag/retriever.py
"""Retriever híbrido BM25 + denso, fusión con Reciprocal Rank Fusion."""

from __future__ import annotations

import json, pickle
from pathlib import Path
from typing import Iterable

import numpy as np
from sentence_transformers import SentenceTransformer

from ml.rag.index import INDEX_DIR, EMBED_MODEL, Doc, _tokenize

RRF_K = 60  # constante estándar

def _rrf(rankings: Iterable[list[int]], k: int = RRF_K) -> list[tuple[int, float]]:
    scores: dict[int, float] = {}
    for rank in rankings:
        for pos, doc_idx in enumerate(rank):
            scores[doc_idx] = scores.get(doc_idx, 0.0) + 1.0 / (k + pos + 1)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)

class HybridRetriever:
    def __init__(self):
        with (INDEX_DIR / "bm25.pkl").open("rb") as f:
            self._bm25 = pickle.load(f)
        self._embeds = np.load(INDEX_DIR / "embeddings.npy")
        with (INDEX_DIR / "docs.jsonl").open() as f:
            self._docs = [Doc(*json.loads(line)) for line in f]
        self._model = SentenceTransformer(EMBED_MODEL)

    def retrieve(self, query: str, k: int = 5) -> list[Doc]:
        # BM25 ranking
        bm25_scores = self._bm25.get_scores(_tokenize(query))
        bm25_rank = list(np.argsort(bm25_scores[::-1]))
        # Dense ranking
        qv = self._model.encode([query], normalize_embeddings=True)[0]
        dense_scores = self._embeds @ qv
        dense_rank = list(np.argsort(dense_scores[::-1]))

        fused = _rrf([bm25_rank[:30], dense_rank[:30]])
        return [self._docs[idx] for idx, _ in fused[:k]]
```

Construir índice y verificar

```
# Asegúrate de tener al menos 3 docs en corpus/ (incluso lorem ipsum sirve para smoke)
mkdir -p ml/rag/corpus/mitre ml/rag/corpus/runbook
cat > ml/rag/corpus/mitre/T1486.md << 'DOC'
# T1486 – Data Encrypted for Impact

Adversaries may encrypt data on target systems...
DOC
cat > ml/rag/corpus/runbook/ransomware_response.md << 'DOC'
# Ransomware response runbook

If file-write entropy > 7.5 and files_touched > 100/min, isolate the host...
DOC

python -m ml.rag.index
# OUTPUT ESPERADO (primer run descarga el modelo BGE – puede tomar 1-2 min):
# indexed 2 docs (BM25 + BGE-1024d)

python - << 'PY'
from ml.rag.retriever import HybridRetriever
r = HybridRetriever()
results = r.retrieve("Process encrypting files in bulk", k=2)
for d in results:
    print(d.id, "→", d.title)
PY
# OUTPUT ESPERADO:
# mitre/T1486.md → T1486 – Data Encrypted for Impact
# runbook/ransomware_response.md → Ransomware response runbook
```

Check (2.5)	Esperado
<code>_index/</code> tiene 3 archivos	sí
Query relevante recupera doc esperado en top-2	sí
Tiempo de retrieval $\leq$ 500ms tras warm-up	sí

2.6 Layer 4 — Triage end-to-end

Código ([ml/llm_triage/triage.py](#))

```

# ml/llm_triage/triage.py
"""
Ensambla: evento → prompt con contexto RAG → LLM → LLMVerdict.

API pública usada por el consumer SOAR:
    verdict = await classify(event)
"""

from __future__ import annotations

import logging
from typing import Optional

from argos_contracts.incident import NormalizedEvent

from ml.llm_triage.client import LLMVerdict, make_client
from ml.rag.retriever import HybridRetriever

logger = logging.getLogger(__name__)

_RETRIEVER: Optional[HybridRetriever] = None
_CLIENT = None

def _retriever() -> HybridRetriever:
    global _RETRIEVER
    if _RETRIEVER is None:
        _RETRIEVER = HybridRetriever()
    return _RETRIEVER

def _client():
    global _CLIENT
    if _CLIENT is None:
        _CLIENT = make_client()
    return _CLIENT

def _build_prompt(event: NormalizedEvent, ctx_docs) -> str:
    ctx = "\n\n".join(f"### {d.title}\n{d.text[:600]}" for d in ctx_docs)
    return (
        f"### Event\n"
        f"Host: {event.host}\n"
        f"MITRE technique: {event.mitre_technique}\n"
        f"Severity: {event.severity}\n"
        f"Layers fired: {event.num_layers_fired}\n"
        f"Confidence (stat): {event.confidence_score}\n"
        f"\n### Context (top RAG docs)\n{ctx}\n"
        f"\n### Task\nClassify the event."
    )

async def classify(event: NormalizedEvent) -> LLMVerdict:
    docs = _retriever().retrieve(

```

```
        f"{event.mitre_technique} {event.host}", k=4
    )
    prompt = _build_prompt(event, docs)
    return await _client().classify(prompt)
```

Probar contra evento sintético

```
python - << 'PY'
import asyncio
from argos_contracts.incident import NormalizedEvent
from argos_contracts.enums import Severity
from ml.llm_triage.triage import classify

evt = NormalizedEvent(
    event_id="evt-llm-001", severity=Severity.MEDIUM,
    mitre_technique="T1486", num_layers_fired=2, confidence_score=0.71,
    host="WIN-VICTIM-01", layer_origin="sigma",
)
print(asyncio.run(classify(evt)))
PY
# OUTPUT ESPERADO:
# LLMVerdict(label='malicious', confidence=0.93, reasoning='...references T1486 and high entropy...')
```

Check (2.6)	Esperado
End-to-end devuelve verdict en ≤ 2s (OpenAI) o ≤ 6s (Llama CPU)	sí
RAG inyecta ≥ 2 docs relevantes en el prompt	sí
Sin context (corpus vacío) → aún devuelve verdict pero <code>confidence < 0.7</code>	sí

Checklist Fase 2

#	Check	OK
1	Synthetic dataset generado (5201 filas)	☐
2	IsoForest + OC-SVM entrenados (3 <code>.joblib</code>)	☐
3	TP rate $\geq$ 85%, FP rate $\leq$ 5% sobre dataset sintético	☐
4	Shannon entropy tests (6) pasan	☐
5	OpenAI client devuelve verdict válido	☐
6	Llama local client devuelve verdict válido	☐
7	RAG indexa corpus y recupera por similitud	☐
8	<code>classify(event)</code> end-to-end funciona en $\leq$ 2s	☐
9	<code>pytest ml/ -q</code> pasa (esperado $\geq$ 20 tests)	☐

FASE 3 — Integración real

3.1 Consumer del stream Wazuh → emitir a `events:normalized`

Qué estás haciendo

Suscribirte al stream de eventos crudos que P3 emite desde Wazuh (`events:raw_wazuh`). Para cada evento, computar features, correr Layer 2, opcionalmente Layer 4, y emitir `NormalizedEvent` al stream que P1 consume (`events:normalized`).

Código central ([ml/consumer.py](#))

```

# ml/consumer.py
"""
Pipeline ML: raw Wazuh event → features → Layer 2 + (opcional) Layer 4 → normalized.

Stream input:  events:raw_wazuh
Stream output: events:normalized (consumido por P1)
Group:        ml-pipeline
"""

from __future__ import annotations

import asyncio, json, logging, os, time, uuid
import redis.asyncio as redis

from argos_contracts.enums import Severity
from argos_contracts.incident import NormalizedEvent
from ml.anomaly.scorer import AnomalyScorer
from ml.llm_triage.triage import classify as llm_classify

logger = logging.getLogger(__name__)

IN_STREAM = "events:raw_wazuh"
OUT_STREAM = "events:normalized"
GROUP = "ml-pipeline"
CONSUMER = os.environ.get("ML_CONSUMER_NAME", "ml-1")

_scorer = AnomalyScorer.load()

def _features_from_wazuh(evt: dict) -> dict:
    """Adaptador del schema Wazuh → features dict para AnomalyScorer."""
    return {
        "syscalls_per_min": evt.get("syscalls_per_min", 0),
        "files_touched_per_min": evt.get("files_touched_per_min", 0),
        "entropy_of_written_bytes": evt.get("entropy_of_written_bytes", 0.0),
        "network_kbps": evt.get("network_kbps", 0),
        "command_line_len": len(evt.get("command_line", "")),
    }

async def _process(r: redis.Redis, raw: dict) -> None:
    evt = json.loads(raw["data"])
    feats = _features_from_wazuh(evt)
    score = _scorer.score(feats)

    if not score.is_anomaly:
        return # sub-threshold → no emit

    norm = NormalizedEvent(
        event_id=f"evt-{uuid.uuid4().hex[:12]}",
        severity=Severity.MEDIUM if score.confidence < 0.85 else Severity.HIGH,
        mitre_technique=evt.get("mitre_technique", "Unknown"),
        num_layers_fired=1, # ML alone; SOAR puede consolidar con otras capas
        confidence_score=score.confidence,

```



```

        host=evt.get("host", "unknown"),
        layer_origin="ml",
    )
    await r.xadd(OUT_STREAM, {"data": norm.model_dump_json()})
    logger.info("ml emit %s host=%s conf=%.2f",
                norm.event_id, norm.host, norm.confidence_score)

async def run() -> None:
    r = redis.from_url(os.environ["REDIS_URL"], decode_responses=True)
    try:
        await r.xgroup_create(IN_STREAM, GROUP, id="0", mkstream=True)
    except redis.ResponseError as e:
        if "BUSYGROUP" not in str(e):
            raise

    while True:
        resp = await r.xreadgroup(GROUP, CONSUMER, {IN_STREAM: ">"},
                                   count=10, block=5000)
        for _, entries in resp or []:
            for entry_id, fields in entries:
                try:
                    await _process(r, fields)
                    await r.xack(IN_STREAM, GROUP, entry_id)
                except Exception:
                    logger.exception("failed to process %s", entry_id)

```

Verificar con eventos inyectados

```

# Terminal 1: arranca consumer ML
python -m ml.consumer

# Terminal 2: simula evento Wazuh
redis-cli XADD events:raw_wazuh '*' data \
    '{"host":"WIN-VICTIM-01","mitre_technique":"T1486","syscalls_per_min":5500,"files_touched_per_min":100}'
# OUTPUT ESPERADO: "1234-0"

# Terminal 1 debería loguear:
# INFO:ml.consumer:ml emit evt-abc123 host=WIN-VICTIM-01 conf=0.93

# Verificar emisión al stream consumido por P1:
redis-cli XLEN events:normalized
# OUTPUT ESPERADO: 1 (o más, si corriste antes)
redis-cli XREVRANGE events:normalized + - COUNT 1
# OUTPUT ESPERADO (JSON):
# 1) 1) "1234-0"
#    2) 1) "data"
#    2) '{"event_id":"evt-abc123","severity":"HIGH",...}'

```

Check (3.1)	Esperado
Consumer ML levanta sin error	sí
Eventos benignos NO se emiten a <code>events:normalized</code>	sí (<code>xlen</code> no crece)
Eventos ransomware-like Sí se emiten	sí
<code>confidence_score</code> correlaciona con severidad del evento	sí

3.2 LLM Triage callable desde el SOAR

P1 lo importa así: `from ml.llm_triage import classify as llm_classify`. Asegúrate de exportarlo desde `ml/llm_triage/__init__.py`:

```
# ml/llm_triage/__init__.py
from ml.llm_triage.triage import classify
__all__ = ["classify"]
```

Y verifica:

```
python -c "from ml.llm_triage import classify; print(classify)"
# OUTPUT ESPERADO:
# <function classify at 0x...>
```

Check (3.2)	Esperado
<code>from ml.llm_triage import classify</code> funciona desde cualquier dir del repo	sí
Si Ollama caído y <code>LLM_BACKEND=llama_local</code> , error captura sin crashear el consumer P1	sí

3.3 Smoke end-to-end con P1

Coordina con P1 una corrida conjunta:

```
# P1 corre:
python -m soar.decision_engine.consumer

# Tú corres:
python -m ml.consumer

# P3 (o tú con un comando manual) inyecta a events:raw_wazuh
# Verifica: incident:* aparece en Redis, notif llega a Telegram con
# layer_origin=ml en los logs.
```

Check (3.3)	Esperado
Evento raw → ML normaliza → SOAR consume → Telegram en ≤ 8s	sí
<code>incident.llm_verdict</code> poblado para T2	sí

Checklist Fase 3

#	Check	OK
1	ML consumer corre y procesa stream	☐
2	Emisión solo cuando is_anomaly	☐
3	Integración con P1 verificada end-to-end	☐
4	LLM verdict atadito al incident en Redis	☐
5	Audit log (P1) refleja <code>layer_origin=ml</code>	☐

FASE 4 — Rehearsal y polish

4.1 Rehearsal de carga (cómo se comporta bajo varios eventos seguidos)

```
# Inyectar 50 eventos en burst
for i in $(seq 1 50); do
  redis-cli XADD events:raw_wazuh '*' data \
    '{"host\":"WIN-VICTIM-01\","mitre_technique\":"T1486\","syscalls_per_min\":"$((RANDOM % 500))}'
done

# Medir cuántos llegaron a events:normalized
sleep 5
redis-cli XLEN events:normalized
# OUTPUT ESPERADO:
# Un número entre 35 y 50 (dependiendo de cuántos pasen el threshold).
# Si < 30, tu ML está descartando demasiado → revisar threshold.
# Si = 50 todos, contamination está muy alta → 1% es razonable.
```

Check (4.1)	Esperado
ML procesa 50 events en $\leq 3s$	sí
No memory leak (RSS estable durante el burst)	sí
Logs no muestran ERROR	sí

4.2 Failover OpenAI → Llama

```
# Simular OpenAI caído (revocar key temporalmente o cambiar a una inválida)
export OPENAI_API_KEY=sk-invalid-test
export LLM_BACKEND=openai_gpt4o_mini

python -m ml.consumer &
redis-cli XADD events:raw_wazuh '*' data '...'
# El consumer P1 debería logear:
# ERROR:soar.decision_engine.consumer:llm classify failed; continuing without enrichment
# Y NO crashearse.

# Switch a Llama local
export LLM_BACKEND=llama_local
# Restart consumer ML – los nuevos eventos enriquecen con Llama.
```

Check (4.2)	Esperado
OpenAI down NO crashea ningún componente	sí
Switch a Llama sin tocar código (solo env var)	sí
Tiempo total de switch $\leq 30s$	sí

4.3 Pre-cache de embeddings

Antes del demo (T-1h), pre-calienta el modelo BGE y deja el HybridRetriever instanciado:

```
python - << 'PY'
from ml.rag.retriever import HybridRetriever
_ = HybridRetriever() # carga modelo + índices
print("RAG retriever ready")
PY
# OUTPUT ESPERADO (primera vez):
# RAG retriever ready (tras 5-15s)
```

Y mantén el proceso vivo (o usa un servidor que cargue una vez).

Checklist Fase 4 — listo para demo

#	Check	OK
1	Burst test pasa con 50 events	☐
2	Failover OpenAI → Llama probado	☐
3	RAG pre-warmed antes del demo	☐
4	Métricas TP/FP registradas en <code>docs/LESSONS_LEARNED.md</code>	☐
5	Rehearsal con P1, P3 cerrado	☐

Apéndice A — Troubleshooting ML/LLM

A.1 `sentence-transformers` descarga falla / lenta

```
# Usar HuggingFace mirror si tu conexión bloquea HF directo
export HF_ENDPOINT=https://hf-mirror.com
# 0 descarga manual del modelo a ~/.cache/huggingface/hub
```

A.2 OpenAI `429 Rate limited`

Reduce `temperature` y/o agrega un sleep entre llamadas. Para el demo es improbable (1 incident a la vez) pero en burst tests sí ocurre.

A.3 Llama responde no-JSON

A veces (Q4 quant) la salida está casi-JSON pero con un prefijo o sufijo. Usa `json.loads` con regex extractor:

```
import re, json
m = re.search(r"\{.*\}", content, re.DOTALL)
parsed = json.loads(m.group(0)) if m else {}
```

A.4 IsoForest predice todo como anomalía

Causa: `contamination` mal calibrado o features no escalados.

Fix: revisa que el `StandardScaler` se haya cargado (no entrenes uno nuevo en inferencia). Verifica `joblib.load(scaler.joblib)` y aplicar a TODOS los features.

A.5 BM25 retorna scores cero

Causa: query tokenizada distinto a docs. Usa siempre `_tokenize` del index.

A.6 Ollama "model not loaded"

```
ollama ps                                # ¿modelo cargado?  
ollama run llama3.1:8b-instruct-q4_K_M "" # forzar carga
```

A.7 Memoria insuficiente con BGE-large

Cambia a BGE-base (~400 MB) en `EMBED_MODEL = "BAAI/bge-base-en-v1.5"` y reconstruye índice. Pierdes ~3 puntos de calidad — aceptable para demo.

Apéndice B — Comandos de emergencia

```
# Reentrenar modelos en frío (5 min)  
python -m ml.data.synthetic_generator --benign 5000 --attack 200  
python -m ml.anomaly.trainer --csv ml/data/synthetic.csv  
  
# Reconstruir índice RAG (1-2 min)  
python -m ml.rag.index  
  
# Forzar Llama (sin tocar P1)  
export LLM_BACKEND=llama_local && systemctl --user restart ml-consumer 2>/dev/null || pkill -f 'ml.c  
  
# Limpiar memoria si Ollama acapara  
ollama stop llama3.1:8b-instruct-q4_K_M  
ollama run llama3.1:8b-instruct-q4_K_M ""  
  
# Resetear consumer group ML  
redis-cli XGROUP DESTROY events:raw_wazuh ml-pipeline  
redis-cli XGROUP CREATE events:raw_wazuh ml-pipeline 0 MKSTREAM
```

Apéndice C — Referencias

Cuando estés en...	Lee
<code>ml/anomaly/</code>	SAD §6.3 (Layer 2), ADR-0001 v2
<code>ml/llm_triage/</code>	SAD §6.5 (Layer 4), ADR-0001 v2
<code>ml/rag/</code>	SAD §6.5.2 (RAG), <code>docs/CONTEXT.md</code> Q3
Calibración	<code>docs/CONTEXT.md</code> (Q3 protocol — semana 2)

Change log

Versión	Fecha	Cambio	Autor
2.0	2026-05-24	Reorganización day-by-day → feature-based (Fase 1-4). Comandos completos, outputs esperados literales, checklists por sección, troubleshooting y emergencia.	P1